

Student notes

What makes working code an engineered system?

Student notes for lecture 1. Every scene shows a situation, asks you to predict, reveals the mechanism, changes one condition, compares the outcomes and names the principle. The pages that follow keep the selected states of each scene with their captions.

Scenes in this lecture

#	SCENE (STUDENT HEADING)	ROLE	DEEP LINK
01	Two green confirmations cannot both be right	Anchor	week-01.html#/two-confirmations
02	The Reserve button is not the whole system	Bridge	week-01.html#/system-boundary
03	The same screen can serve one person and exclude another	Anchor	week-01.html#/stakeholder-lenses
04	Faster is not a complete description of better	Bridge	week-01.html#/quality-tradeoffs
05	A failure can teach the next requirement	Anchor	week-01.html#/feedback-lifecycle
06	What would make you believe this works?	Bridge	week-01.html#/evidence-not-confidence
07	Who carries the cost of an engineering shortcut?	Bridge	week-01.html#/responsibility-and-impact
08	One domain, many kinds of explanation	Bridge	week-01.html#/course-evidence-map

Campus Rooms is a fictional room-booking service; every name, number and incident is a teaching example unless a source is named. Rules and values marked example assumption belong to that example: they are illustrative, not measurements or real policies.

The screen says **Confirmed**. Is that enough?

Campus Rooms is a fictional service that reserves campus rooms. Every name, number and incident in this course is a teaching example.

The situation

Ari (U-01) reserves room C101 for 10:00–11:00. The screen answers:
Confirmed: C101, 10:00–11:00. Your booking is saved. A confirmation message is on its way.

The question of this week

What would you need to see before you trust this message? (A) The green tick and the word Confirmed. (B) The stored calendar for the room. (C) What happens at the door at 10:00.

About the examples

Campus Rooms is a fictional room-booking service. Every name, number and incident in this course is a teaching example unless a source is named on the page. Room C101 and the 10:00 to 11:00 hour recur through the whole course.

What you already know, and where this lecture goes

What makes working code an engineered system?

Start from a program you know

Think of a program you have written or used every week. What does it do when everything goes as planned? What could make it fail outside that happy path: a second user, a lost connection, a rule that changes, a wrong assumption? Who else is affected when it fails?

Where this applies

A throwaway calculation on your own machine needs none of this. A shared booking service that other people depend on needs all of it. The care you take grows with the consequences of a failure and with how much you do not know yet.

The thread that runs through the course

Every habit in this course, a precise rule, an independent check, a bounded permission, a reviewable change, is also what it takes to direct and check an automated engineering assistant. Weeks 2, 7, 8, 9, 10 and 12 each return to that for a moment; Week 13 looks at it in full.

The order of the weeks

The fourteen weeks are an order for learning, not fourteen separate phases of a project. A real project comes back to requirements, design, checks and operation again and again.

By the end of this lecture you can

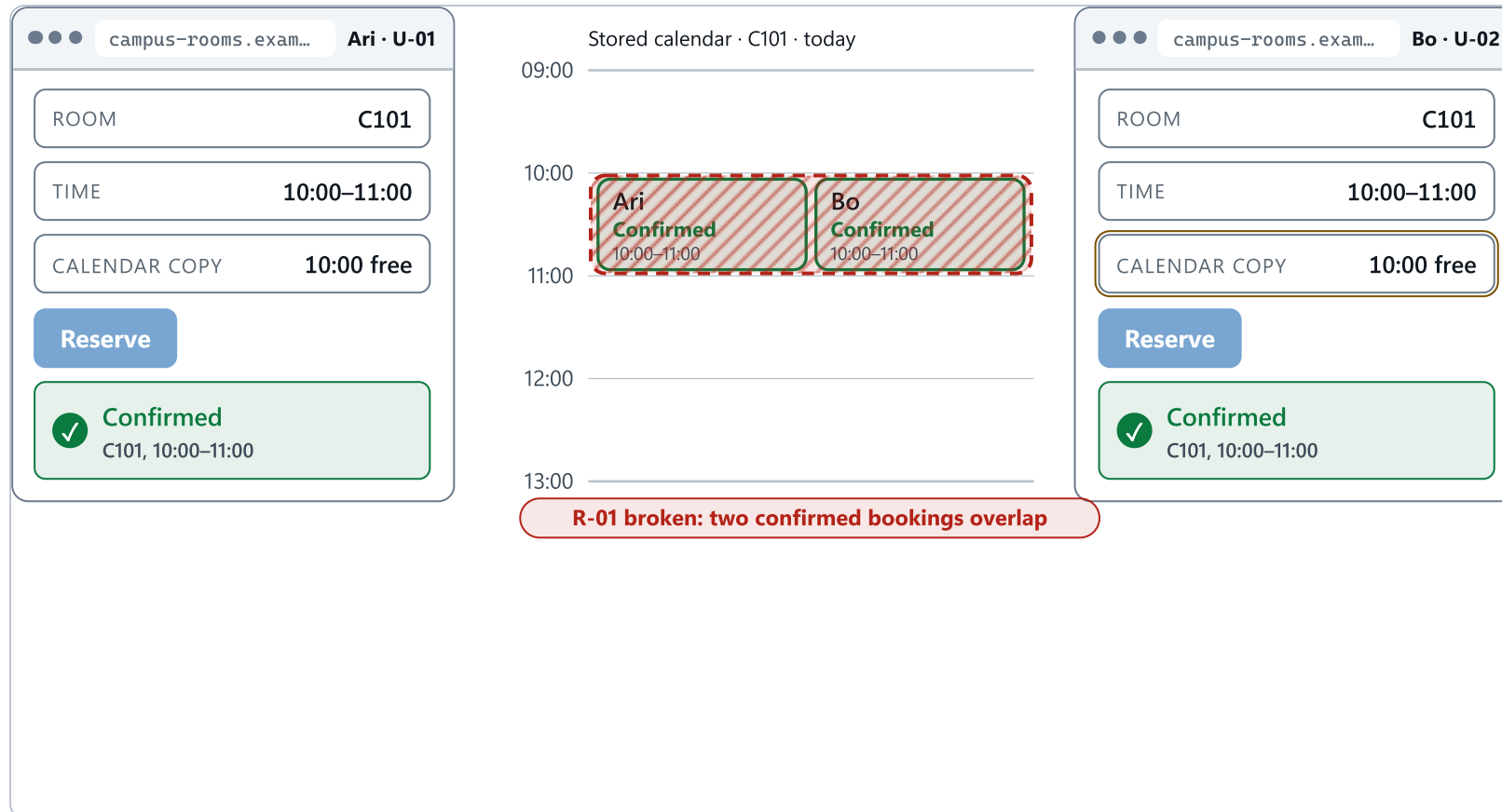
- 1 Distinguish a locally correct function from a dependable user-facing service.
- 2 Identify stakeholders, system boundaries, and competing quality concerns.
- 3 Explain why delivery and maintenance belong in the engineering lifecycle.
- 4 Separate an observation, an assumption, and evidence supporting a claim.
- 5 Explain one technical decision in terms of benefit, cost, and affected people.

Two green confirmations cannot both be right

Panel 1 of 3 · Two confirmations on screen, two overlapping bookings stored

Condition: Baseline: each screen checks its own copy of the room calendar and decides alone State: 5 of 7

The stored calendar holds **two confirmed bookings for C101 in the same hour**. Rule R-01 ([EXAMPLE ASSUMPTION](#)): a room has no overlapping confirmed bookings. The stored record breaks it, even though each screen looked right.



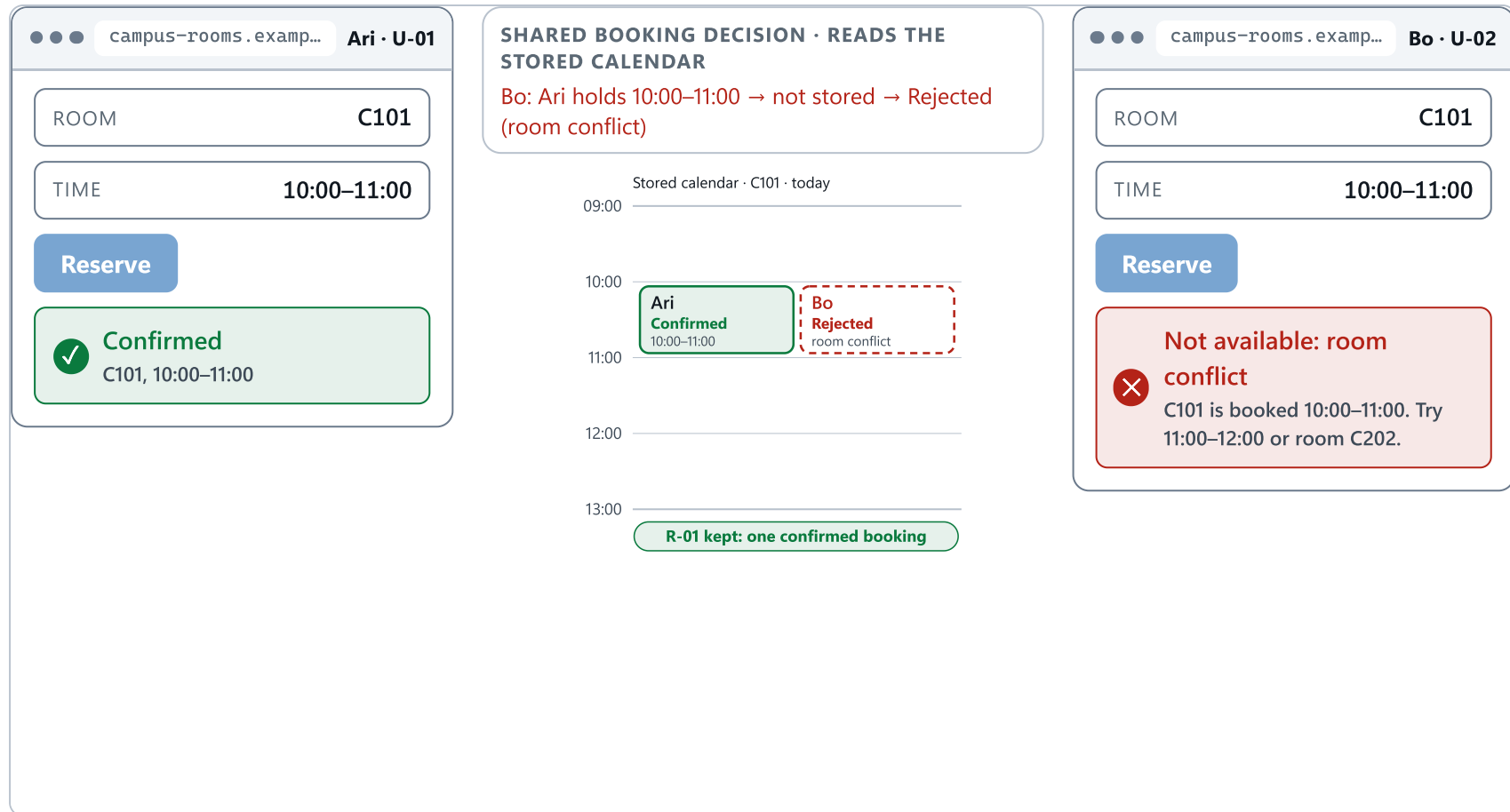
Rule R-01, no overlapping confirmed bookings for the same room, is broken in the stored calendar although each screen showed success. Each screen decided from its own copy of the calendar, and Bo's copy was stale.

Two green confirmations cannot both be right

Panel 2 of 3 · One shared decision: one confirmation, one clear rejection

Condition: Changed condition: one shared decision reads the stored calendar before any screen says Confirmed State: 3 of 5

Bo clicks Reserve. The shared decision finds Ari's booking in the same hour and answers **not available: room conflict**. Bo's screen shows the rejection and offers 11:00–12:00 or room C202.



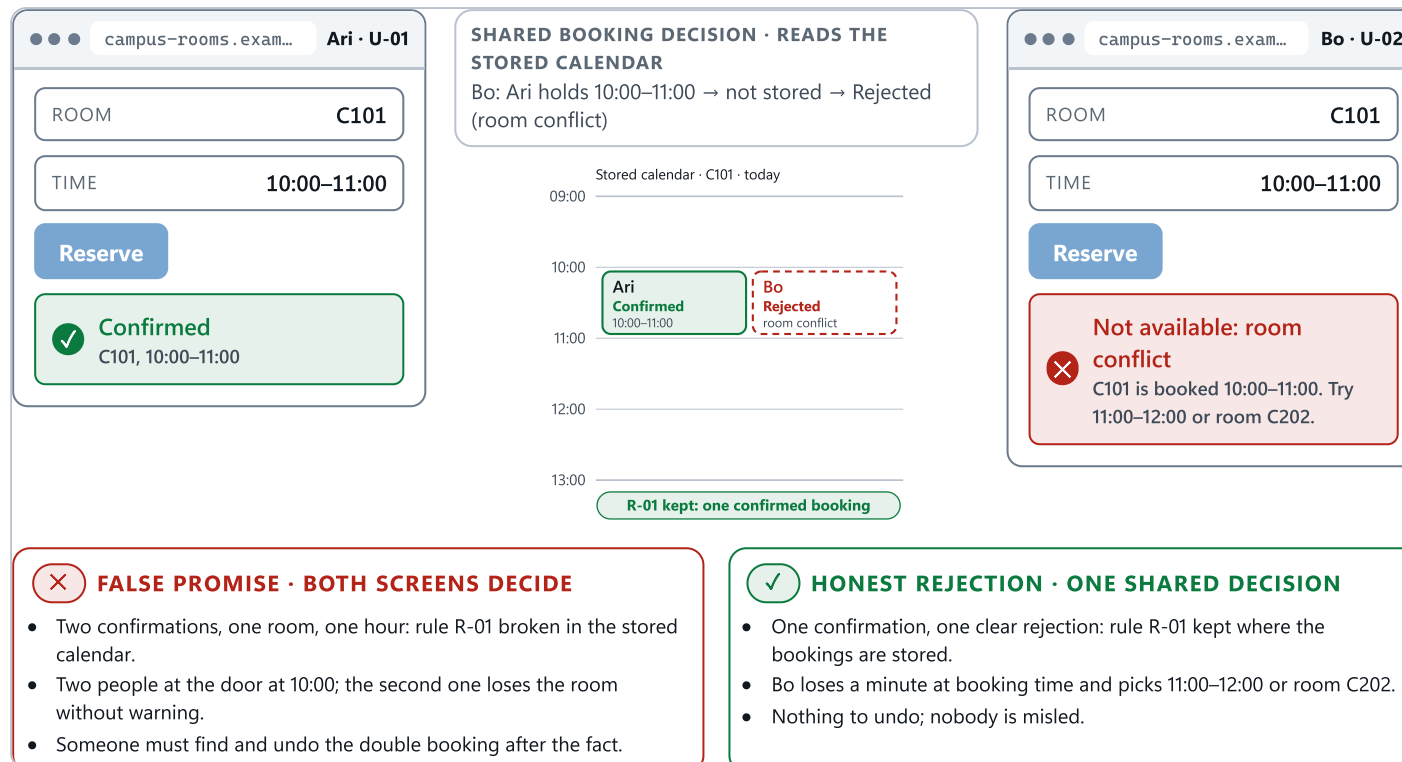
Rule R-01 is kept: the shared decision reads the stored calendar before any screen says Confirmed. Bo sees the reason and two alternatives.

Two green confirmations cannot both be right

Panel 3 of 3 · Compare the consequences

Condition: **Changed condition: one shared decision reads the stored calendar before any screen says Confirmed** State: 4 of 5

Compare the two outcomes. A false promise sends two people to one door. An honest rejection costs Bo a minute at booking time and misleads nobody. How the shared decision stays correct when both requests arrive in the same instant is the question for Week 6.



How the shared decision stays correct when both requests arrive in the same instant is the question for Week 6.

Principle · Engineering judges behavior against an intended outcome and its consequences, not appearance alone.

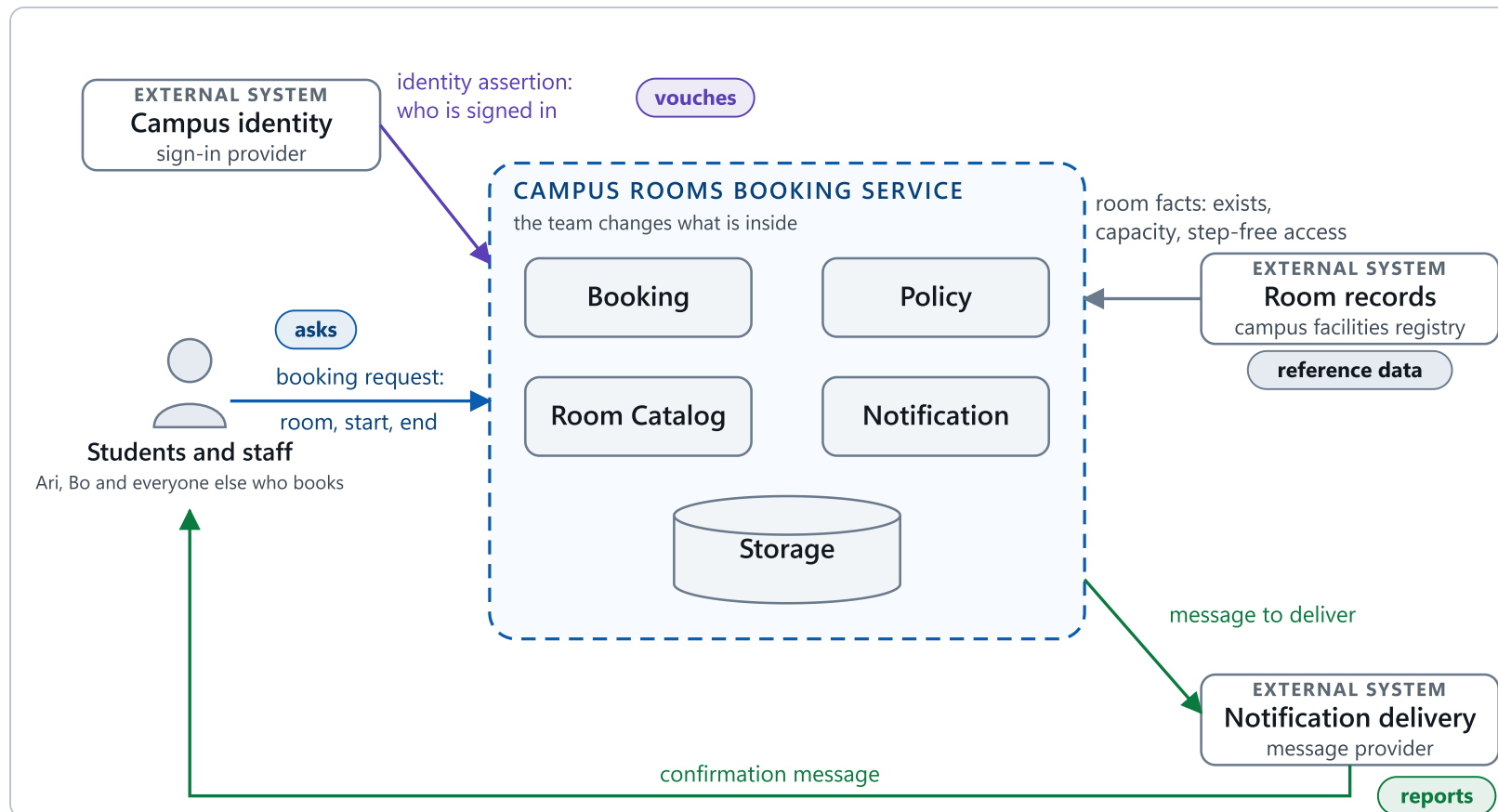
In short · Judge what the system does, not what the screen says.

The Reserve button is not the whole system

Panel 1 of 2 · The context map with every interaction labeled

Condition: Baseline: every external system available State: 6 of 7

A booking request, an identity assertion and a notification are three kinds of information. One **asks**, one **vouches** for a fact, one **reports** a result. The service treats them differently: it decides on the request, trusts the assertion within limits, and hands the report to a provider.



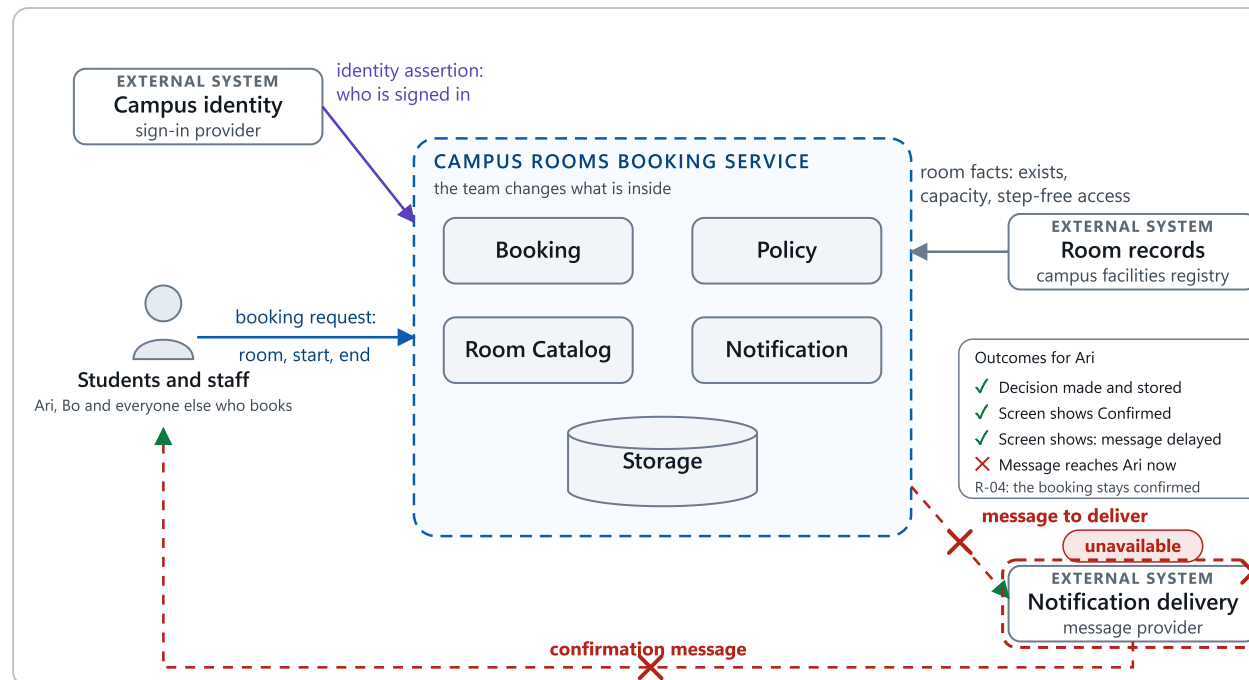
Inside the dashed boundary: Booking, Policy, Room Catalog, Notification and Storage, the modules the team changes. Outside: students and staff, campus identity, room records and notification delivery.

The Reserve button is not the whole system

Panel 2 of 2 · One dependency unavailable: what remains possible

Condition: Changed condition: notification delivery unavailable; room storage unchanged State: 2 of 3

Still possible: the decision is made and stored; Ari's screen shows Confirmed and says that the message is delayed. Not possible right now: the message reaching Ari. Rule R-04 **EXAMPLE ASSUMPTION**: a failed notification does not undo a confirmed booking, and the pending message stays visible and recoverable.



Notification delivery is marked with a broken line and the word unavailable. Still possible: The booking decision is made and stored; Ari's screen shows Confirmed; Ari's screen shows that the confirmation message is delayed. Not possible now: The confirmation message reaches Ari now (rule R-04).

Principle · A system boundary identifies responsibility and dependencies; it does not make external behavior disappear.

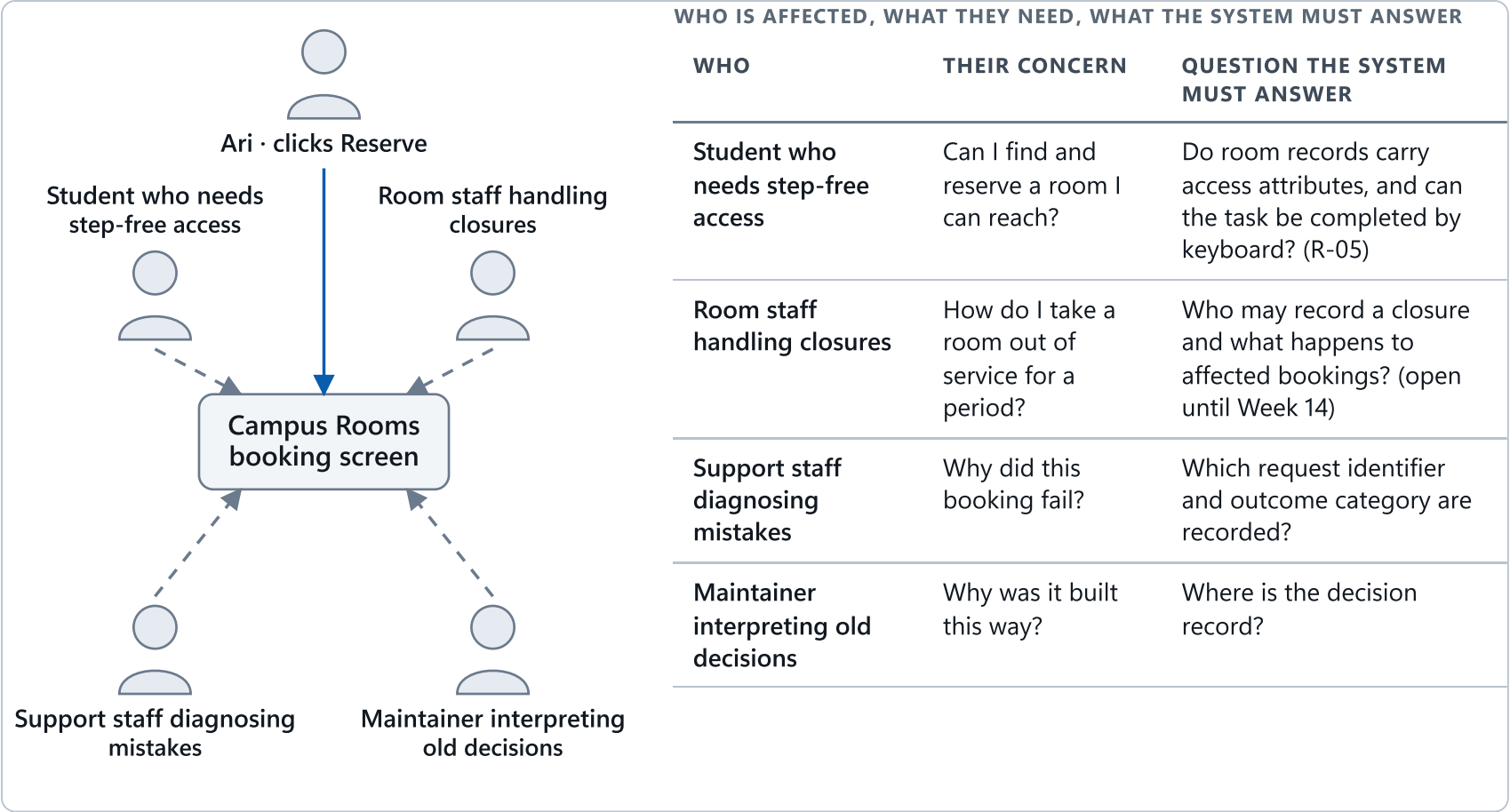
In short · What sits outside your boundary still reaches your users.

The same screen can serve one person and exclude another

Panel 1 of 3 · Who is affected and what the system must answer

Condition: Baseline: the polished booking screen, used with a mouse State: 6 of 7

Each concern became a question the system must answer: about room data, closures, records and decisions. Room closures stay an open question until Week 14.



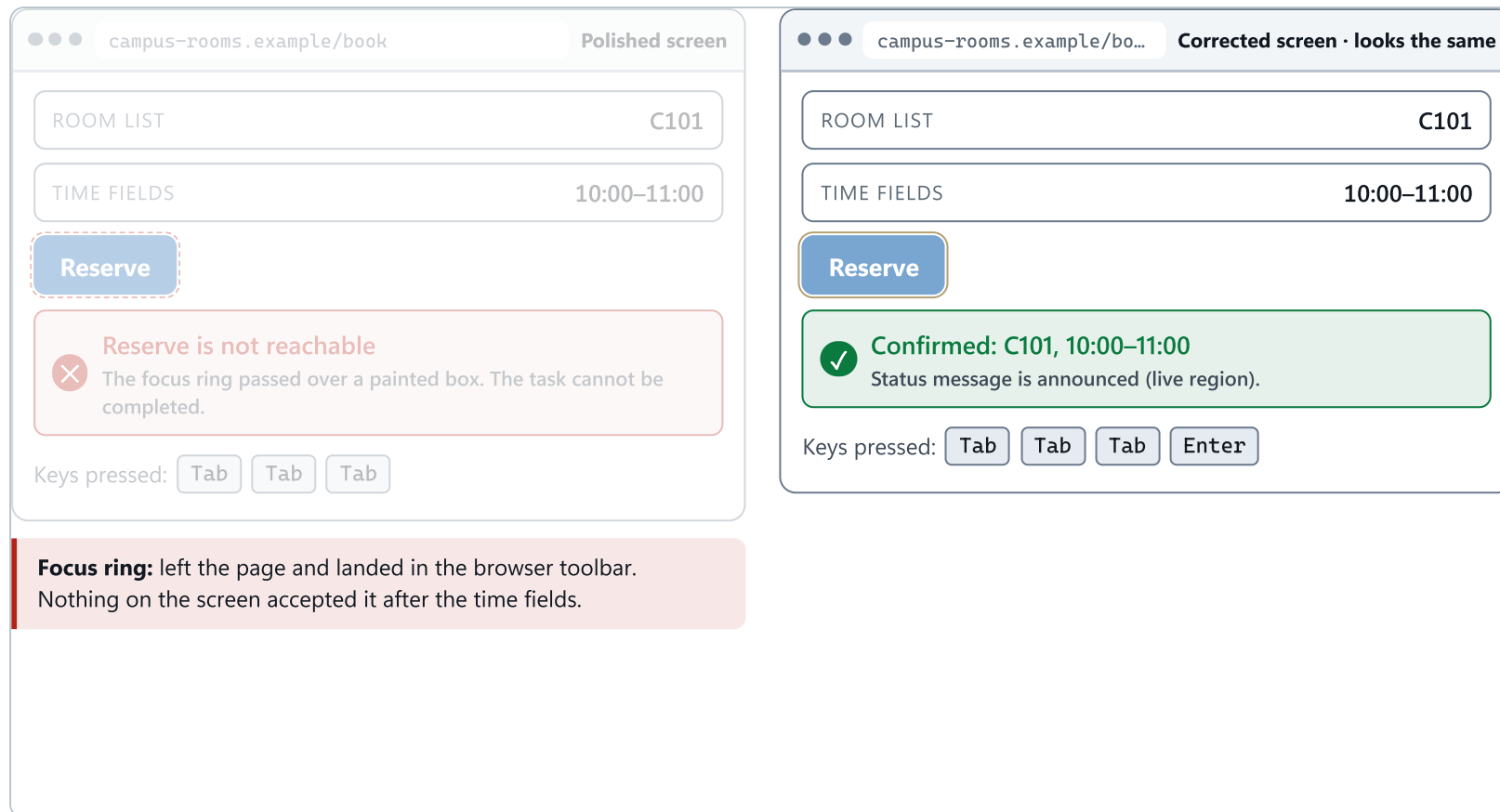
Room closures stay an open need until Week 14: who may record a closure, and what happens to affected bookings.

The same screen can serve one person and exclude another

Panel 2 of 3 · The keyboard route that completes the booking

Condition: Changed condition: the same screen, used with the keyboard only State: 7 of 8

Enter → Confirmed, and the status message is announced, so a screen reader speaks it too. Rule R-05 **EXAMPLE ASSUMPTION**: the core task can be completed by keyboard, with status and error information you can perceive.



The failed route on the polished screen: Tab to room list; Tab to time fields; Reserve control is not reachable; task cannot be completed. The corrected route: Tab to room list; Tab to time fields; Tab to Reserve; Enter; Status message is announced. Compare on task completion, not appearance.

The same screen can serve one person and exclude another

Panel 3 of 3 · Same appearance, different service: accessibility

Condition: Changed condition: the same screen, used with the keyboard only State: 8 of 8

Compare on **task completion, not appearance**: the polished screen completes 0 of 1 bookings by keyboard, the corrected screen 1 of 1. Same appearance, different service.

campus-rooms.example/book

Polished screen

ROOM LISTC101

TIME FIELDS10:00–11:00

Reserve

✖

Reserve is not reachable

The focus ring passed over a painted box. The task cannot be completed.

Keys pressed:

Tab

Tab

Tab

campus-rooms.example/bo...

Corrected screen · looks the same

ROOM LISTC101

TIME FIELDS10:00–11:00

Reserve

✓

Confirmed: C101, 10:00–11:00

Status message is announced (live region).

Keys pressed:

Tab

Tab

Tab

Enter

COMPARE ON TASK COMPLETION, NOT APPEARANCE

SCREEN	APPEARANCE	MOUSE	KEYBOARD ONLY
Polished screen	identical	1 of 1 completed	0 of 1 completed
Corrected screen	identical	1 of 1 completed	1 of 1 completed

THE NAME FOR THIS DIFFERENCE

Accessibility: you can complete the task however you use the service, with a mouse, the keyboard only or a screen reader. It is part of whether the service works.

Reference: W3C, Web Content Accessibility Guidelines 2.2 (S25)

The two screens look identical and complete the booking with a mouse. With the keyboard only, one of them completes it and the other cannot. That difference is accessibility, and it is part of whether the service works.

Principle · Quality depends on the people and conditions included in the definition of success.

In short · If someone cannot finish the task, it does not work.

Faster is not a complete description of better

Panel 1 of 2 · Two designs under the stated constraints

Condition: Baseline: response-time target: an answer within 2 s; correctness (R-01) required State: 7 of 7

Under a 2 s target both designs are fast enough. Correctness is a condition of acceptance, so Design 1 is out before speed is compared. Faster described one attribute; better needs the stated constraints.

QUALITY ATTRIBUTES OF TWO DESIGNS (EXAMPLE ASSUMPTIONS)

QUALITY ATTRIBUTE	DESIGN 1 · THE SCREEN DECIDES	DESIGN 2 · ONE SHARED DECISION
Response time until an answer appears EXAMPLE ASSUMPTION	about 0.1 s	about 0.4 s
Correctness when two requests overlap (R-01) condition of acceptance	✗ two confirmations possible	✓ one confirmation, one rejection
Recoverability after a wrong confirmation	✗ someone must find and undo the double booking later	✓ nothing to undo
Privacy of other people's bookings	✗ every screen holds a copy of the whole room calendar	✓ the screen sends its request; only the decision reads the calendar
Change effort when the booking rule changes	✗ every copy of the rule in every screen changes	✓ one decision point changes

STATED CONSTRAINTS EXAMPLE ASSUMPTIONS

Response-time target: an answer within 2 s.
Correctness (R-01): required in every case, not traded for speed.

THE STANDARD VOCABULARY

ISO/IEC 25010:2023 lists nine product quality characteristics, among them performance efficiency, reliability, security, interaction capability and maintainability.

The rows of this table, in those names:

Response time	performance efficiency
Correctness	functional suitability
Recoverability	reliability
Privacy	security
Change effort	maintainability

Example assumption: illustrative values for two fictional designs, not measurements of a real service

Read the rows as the reasoning: Design 1 fails correctness, a condition of acceptance, so its speed never decides. The response times and the targets are example assumptions for two fictional designs.

Principle · Compare alternatives under stated constraints; some requirements are conditions of acceptance rather than points in a score.

In short · Some requirements are must-haves, not points to trade.

Faster is not a complete description of better

Panel 2 of 2 · The same designs under a 0.3 s target

Condition: Changed condition: response-time target: an answer within 0.3 s; correctness (R-01) still required State: 5 of 5

Two options keep the rule and deal with the target; the first hides a delay by giving up the rule. An optimization that preserves the rule is engineering; one that hides the delay is a return to Design 1.

QUALITY ATTRIBUTES OF TWO DESIGNS (EXAMPLE ASSUMPTIONS)

QUALITY ATTRIBUTE	DESIGN 1 · THE SCREEN DECIDES	DESIGN 2 · ONE SHARED DECISION
Response time until an answer appears EXAMPLE ASSUMPTION	about 0.1 s	about 0.4 s
Correctness when two requests overlap (R-01) condition of acceptance	✗ two confirmations possible	✓ one confirmation, one rejection
Recoverability after a wrong confirmation	✗ someone must find and undo the double booking later	✓ nothing to undo
Privacy of other people's bookings	✗ every screen holds a copy of the whole room calendar	✓ the screen sends its request; only the decision reads the calendar
Change effort when the booking rule changes	✗ every copy of the rule in every screen changes	✓ one decision point changes

STATED CONSTRAINTS EXAMPLE ASSUMPTIONS

Response-time target: an answer within 0.3 s.
Correctness (R-01): required in every case, not traded for speed.

OPTIONS UNDER THE 0.3 S TARGET

OPTION	RULE	EFFECT
Answer Confirmed before the shared decision returns	breaks R-01	hides the delay and brings back two confirmations
Look up the room's day faster: index the stored bookings by room and day	keeps R-01	about 0.25 s in the example; the shared decision still decides
Show Checking availability, then the answer	keeps R-01	the wait is visible; nobody is misled

Example assumption: illustrative values for two fictional designs, not measurements of a real service

Correctness (R-01) stays required. Two options keep the rule; the first hides the delay by giving up the rule.

Principle · Compare alternatives under stated constraints; some requirements are conditions of acceptance rather than points in a score.

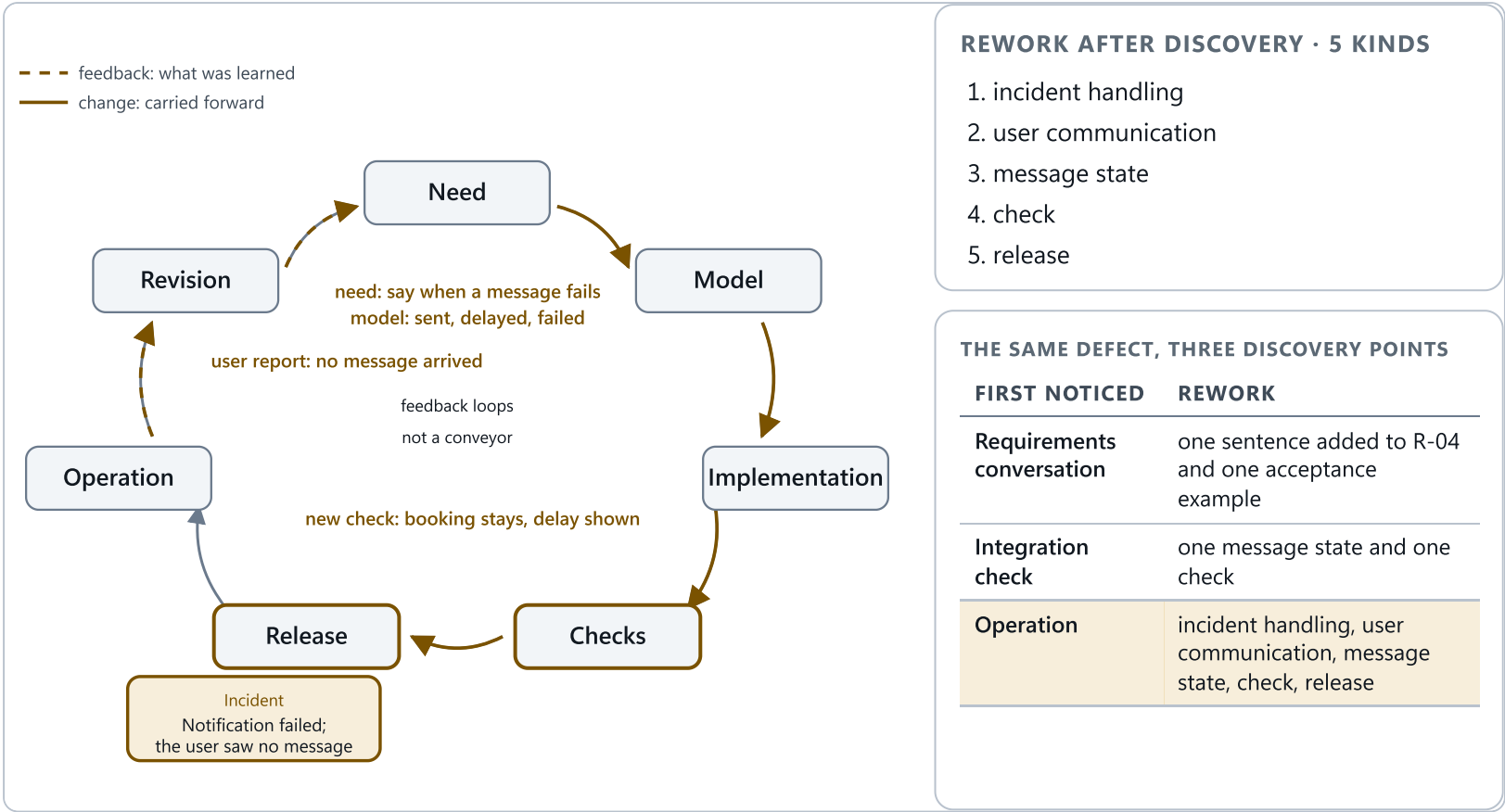
In short · Some requirements are must-haves, not points to trade.

A failure can teach the next requirement

Panel 1 of 3 · The loop and the feedback path of a defect found in operation

Condition: Baseline: the missing message is first noticed in operation, by a user State: 6 of 7

Release. The change ships. Count the rework: incident handling, user communication, message state, check, release. Five kinds, and the user found the defect.

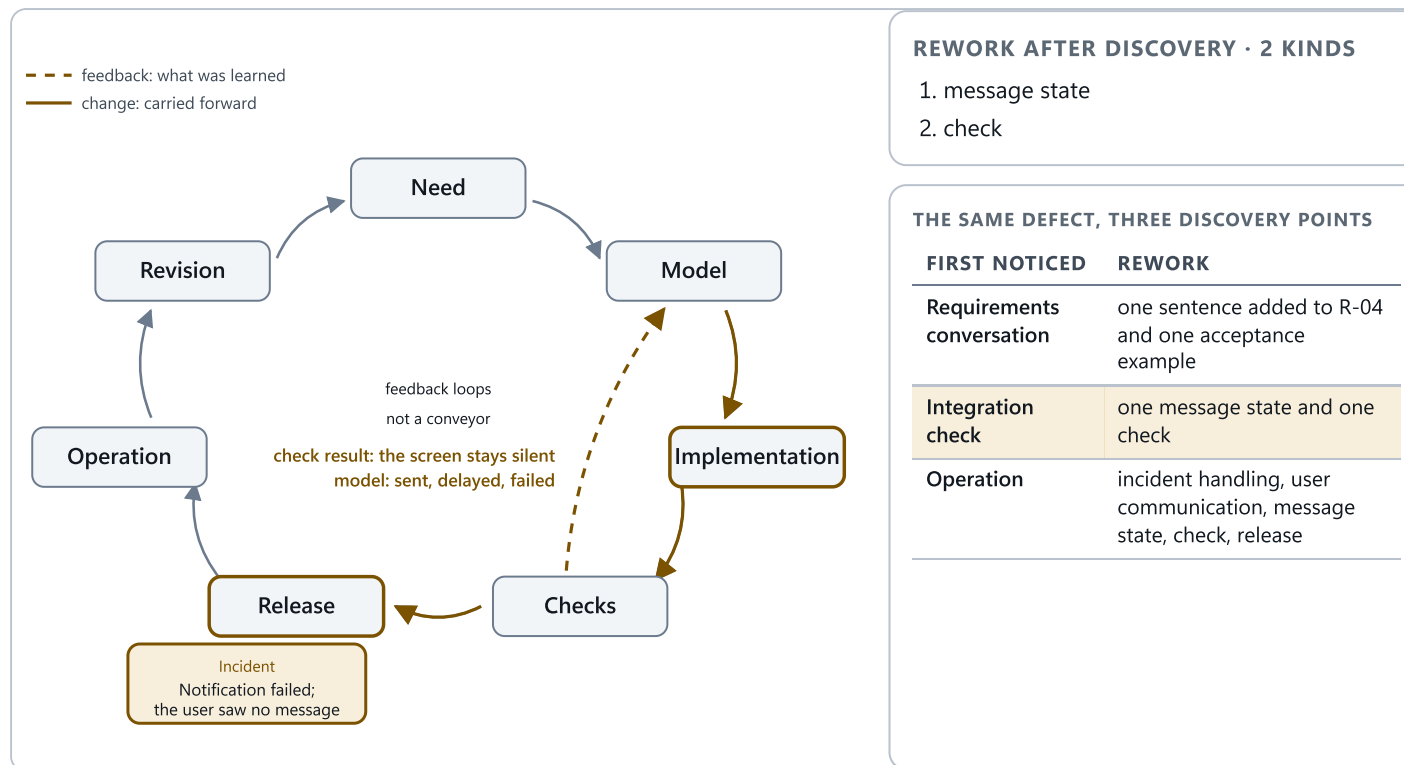


A failure can teach the next requirement

Panel 2 of 3 · The feedback path of the same defect found at an integration check

Condition: **Changed condition:** the same defect is first noticed at an integration check State: 4 of 4

Release with the message state in place. Compare the rework list with the baseline.



Path: Checks → Model → Implementation → Checks → Release. Dashed steps carry what was learned back into the loop; solid steps carry the change forward.
Rework: one message state and one check, against five kinds when a user found the defect.

Principle · The lifecycle is a network of feedback loops, not a one-way conveyor ending when code runs.

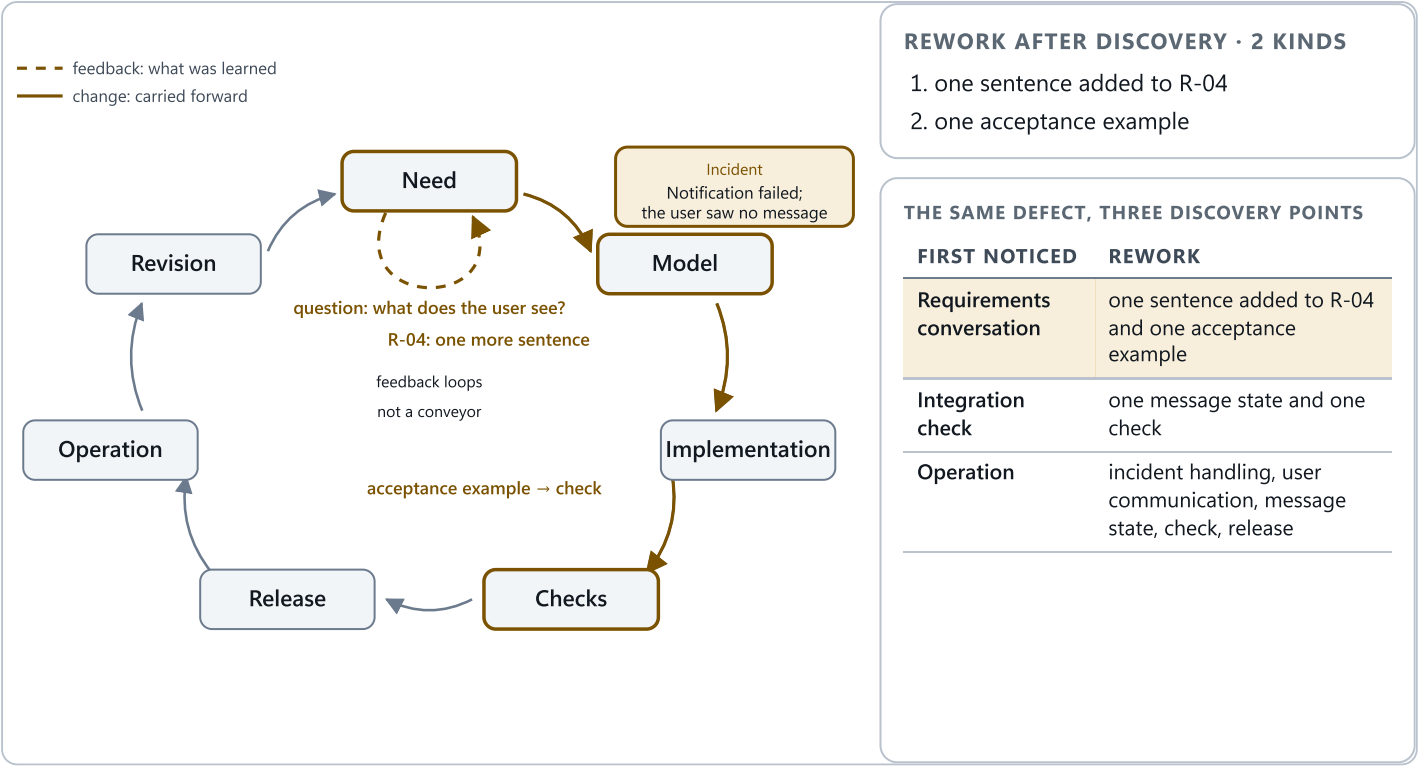
In short · The earlier you learn, the less you redo.

A failure can teach the next requirement

Panel 3 of 3 · The feedback path of the same defect found while talking about the need

Condition: Changed condition: the same defect is first noticed in the requirements conversation State: 3 of 3

The example flows into **Model** and **Checks** as they are written. Compare the three rework lists: the same defect, three different amounts of undoing.



Path: Need → Model → Implementation → Checks. Dashed steps carry what was learned back into the loop; solid steps carry the change forward. Rework: one sentence added to R-04 and one acceptance example. Iteration revisits understanding; an increment adds usable capability.

Principle · The lifecycle is a network of feedback loops, not a one-way conveyor ending when code runs.

In short · The earlier you learn, the less you redo.

What would make you believe this works?

Panel 1 of 1 · Claim, check and remaining uncertainty, with the counterexample

Condition: Changed condition: workload: two people request the same room in the same moment State: 3 of 3

The third claim named its conditions, and its checks include the two-at-once case at the storage boundary, so it covers the new workload. The habit of this course: state the evidence and its limits together.

CLAIM	CHECK THAT SUPPORTS IT	REMAINING UNCERTAINTY
It looks right	✓ Someone read the screen after one booking	? Nothing about the stored calendar, other people or other rooms
It passed one example	✓ Ari books C101 10:00–11:00 on an empty calendar; the stored calendar shows one booking	? Other times, other rooms, a booking that already exists, two people at once exposed assumption: the assumption that only one person books at a time
It met the stated checks under these conditions	✓ Interval cases (overlap, adjacent, other room) and the two-at-once case at the storage boundary covers the new workload	? Conditions outside the stated list: three people at once, a lost response, a clock that differs
Counterexample: Ari and Bo both request C101 10:00–11:00 in the same moment · exposes the assumption that only one person books at a time		

Counterexample: Ari and Bo both request C101 10:00–11:00 in the same moment. It exposes the assumption that only one person books at a time; the single example still holds for one person at a time.

Principle · A check supports a bounded claim; confidence without scope is difficult to evaluate.

In short · A test shows what it tested. Say what it covered.

Who carries the cost of an engineering shortcut?

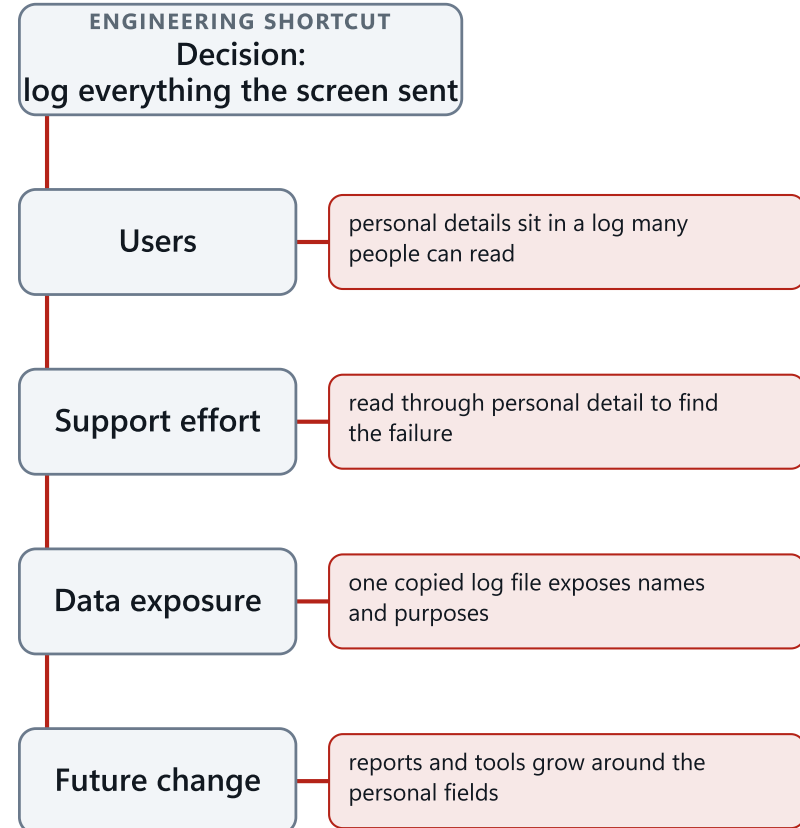
Panel 1 of 2 · The log as first written, and the consequences it imposes

Condition: Baseline: support question: why did this booking fail? State: 5 of 7

Future change. Logging everything: reports and tools grow around the personal fields. Minimal record: a field is added when a question needs it.

SUPPORT LOG ENTRY AS FIRST WRITTEN

request_id	req-4f9c	needed
time	09:41:07	needed
outcome	failed: storage timeout	needed
user	Ari (U-01)	not needed
room and slot	C101, 10:00-11:00	not needed
purpose	counselling session	not needed
full request	every field the screen sent	not needed



Red: what logging everything imposes; the minimal record removes each one.

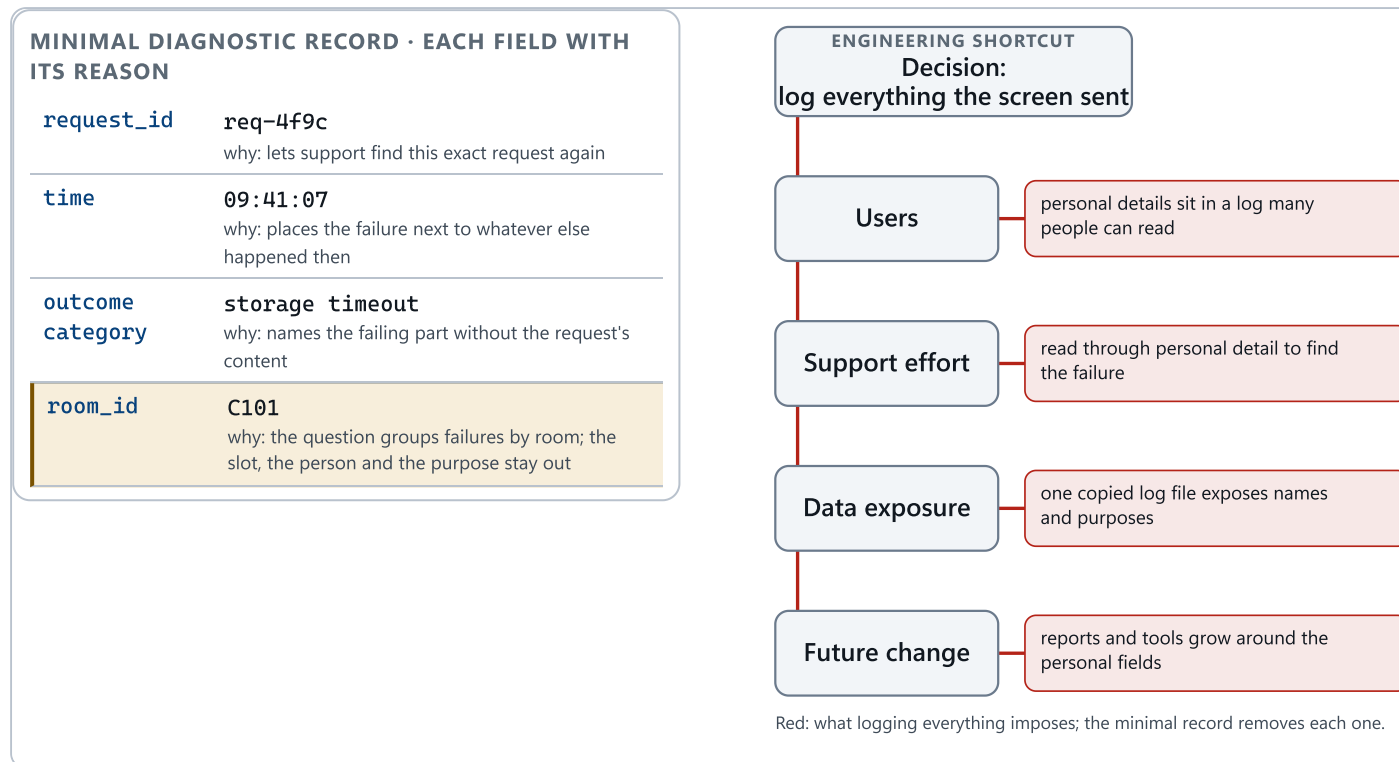
Fields marked not needed describe the person, not the failure. Each red box is a consequence for people who did not choose the logging design.

Who carries the cost of an engineering shortcut?

Panel 2 of 2 · The minimal diagnostic record with the rationale for each field

Condition: Changed condition: support question: is the failure specific to one room? State: 2 of 3

room_id = C101 is added, with its reason: the question groups failures by room; the slot, the person and the purpose stay out. Deliberate collection: one field, one written reason. Collecting everything by default needs no reason and answers no question better.



The changed support question justified exactly one more field, **room_id**, with its reason. The slot, the person and the purpose stay out of the log.

Principle · Professional responsibility includes the consequences imposed on people who did not make the technical choice.

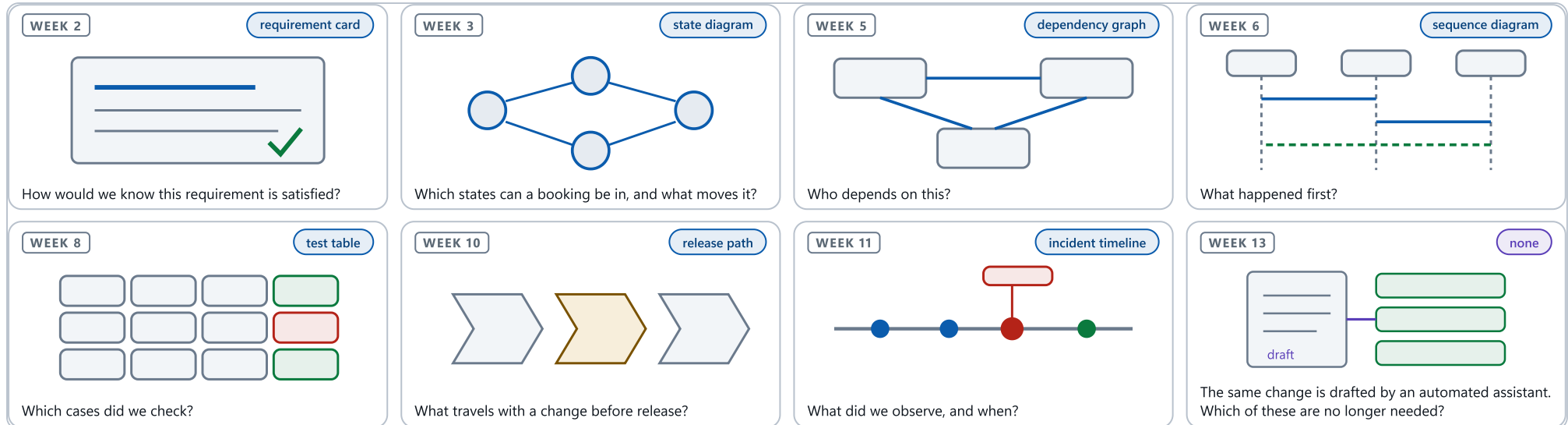
In short · Your shortcut becomes someone else's cost.

One domain, many kinds of explanation

Panel 1 of 2 · The course map: question to representation

Condition: **Baseline:** question: who depends on the booking decision? State: 5 of 7

Week 11: What did we observe, and when? An incident timeline. **Week 13:** The same change is drafted by an automated assistant. Which of these are no longer needed? None: every representation and check above still applies.



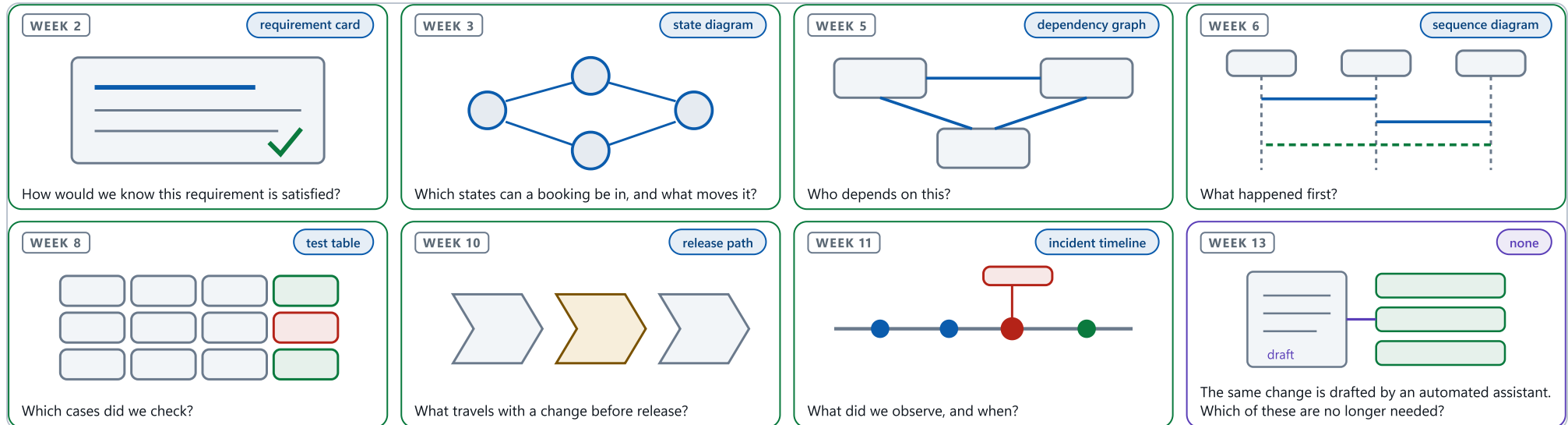
Week 2: How would we know this requirement is satisfied? → requirement card with its acceptance example. Week 3: Which states can a booking be in, and what moves it? → state diagram. Week 5: Who depends on this? → dependency graph. Week 6: What happened first? → sequence diagram. Week 8: Which cases did we check? → test table. Week 10: What travels with a change before release? → release path. Week 11: What did we observe, and when? → incident timeline. Week 13: The same change is drafted by an automated assistant. Which of these are no longer needed? → none: every representation and check above still applies.

One domain, many kinds of explanation

Panel 2 of 2 · The actor changes; the representations and checks stay

Condition: **Changed condition:** actor: the booking change is drafted by an automated assistant, not a colleague State: 2 of 2

None. The requirement card, the state diagram, the dependency graph, the sequence, the test table, the release path and the incident timeline all still apply, because they describe the system and its behavior, not who typed the draft. Week 13 examines this in full.



A change drafted by an automated assistant still needs the requirement card, the state diagram, the dependency graph, the sequence, the test table, the release path and the incident timeline. Week 13 examines this in full.

Principle · Choose a representation for the question, not a diagram style for the whole course.

In short · Pick the diagram that answers your question.

Eight principles, four checks

One principle per scene. Then four questions: decide your answer before you open it.

Principles

1 · ANCHOR

Engineering judges behavior against an intended outcome and its consequences, not appearance alone.

In short: Judge what the system does, not what the screen says.

[week-01.html#/two-confirmations](#)

2 · BRIDGE

A system boundary identifies responsibility and dependencies; it does not make external behavior disappear.

In short: What sits outside your boundary still reaches your users.

[week-01.html#/system-boundary](#)

3 · ANCHOR

Quality depends on the people and conditions included in the definition of success.

In short: If someone cannot finish the task, it does not work.

[week-01.html#/stakeholder-lenses](#)

4 · BRIDGE

Compare alternatives under stated constraints; some requirements are conditions of acceptance rather than points in a score.

In short: Some requirements are must-haves, not points to trade.

[week-01.html#/quality-tradeoffs](#)

5 · ANCHOR

The lifecycle is a network of feedback loops, not a one-way conveyor ending when code runs.

In short: The earlier you learn, the less you redo.

[week-01.html#/feedback-lifecycle](#)

6 · BRIDGE

A check supports a bounded claim; confidence without scope is difficult to evaluate.

In short: A test shows what it tested. Say what it covered.

[week-01.html#/evidence-not-confidence](#)

7 · BRIDGE

Professional responsibility includes the consequences imposed on people who did not make the technical choice.

In short: Your shortcut becomes someone else's cost.

[week-01.html#/responsibility-and-impact](#)

8 · BRIDGE

Choose a representation for the question, not a diagram style for the whole course.

In short: Pick the diagram that answers your question.

[week-01.html#/course-evidence-map](#)

Checks: decide your answer before reading the one below each question

1. What is wrong with two successful overlapping confirmations?

They violate the intended exclusivity rule even when each screen and request appears successful.

2. Why include support staff in the stakeholder map?

Their diagnostic needs influence observability, documentation, privacy, and recovery decisions.

3. Does passing one booking example establish correctness for simultaneous requests?

No. The evidence does not cover that interaction condition.

4. Which representation answers: what happens after a timeout?

A state model or a sequence/timeline view, depending on whether the question concerns possible states or a particular interaction.

What carries forward, and where to read more

Next week starts from one sentence and turns it into requirements you can check.

Week 2 starts from this request:

“Make room booking fair and easy.”

Carried forward to Week 2

- The context map: the booking service inside its boundary, four dependencies outside.
- The no-overlap concern: one confirmed booking per room and hour (R-01), kept where bookings are stored.
- Open assumptions: who may record a room closure; what the user sees while a message is delayed; how the shared decision behaves when two requests arrive in the same instant.

In practice · examples from 2026

Teams keep requirements and incident reports in issue trackers such as GitHub Issues or Jira, check accessibility with axe or Lighthouse next to a keyboard-only walk-through, and draw context maps in Mermaid or draw.io.

Sources for this lecture

- S01** IEEE Computer Society, Guide to the Software Engineering Body of Knowledge (SWEBOK V4.0a) <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- S02** ACM / IEEE-CS / AAAI, Computer Science Curricula 2023, final report <https://csed.acm.org/final-report/>
- S24** Software Engineering at Google, chapter 1: What is software engineering? <https://abseil.io/resources/swe-book/html/ch01.html>
- S25** W3C, Web Content Accessibility Guidelines 2.2 <https://www.w3.org/TR/WCAG22/>
- S46** ISO/IEC 25010:2023, SQuaRE product quality model <https://www.iso.org/standard/78176.html>

Terms used this week

Software engineering

Making and changing software that people depend on, over its whole lifetime: agreeing what it must do, building, checking, running and revising it, with the consequences for other people counted in. It is about change, coordination and quality.

Stakeholder

A person or group affected by the service: the person booking, room staff, support staff, a future maintainer.

Accessibility

You can complete the task however you use the service: a mouse, the keyboard only or a screen reader. Part of whether the service works.

Requirement

A statement of what the service must do or keep true, written so that a check can show whether it holds. R-01 is one.

Constraint

A condition every acceptable design must meet, such as correctness under overlapping requests.

Quality attribute

A property of how the service behaves: response time, recoverability, privacy, change effort. ISO/IEC 25010:2023 groups them into nine product quality characteristics.

Evidence

What a check showed, together with the conditions it ran under. Its scope is part of the evidence.

Lifecycle

The loop of need, model, implementation, checks, release, operation and revision that a service goes around repeatedly.

Trade-off

A choice between alternatives under stated constraints, where each option costs something the other keeps.